Boosting the Parallel Efficiency of Chess Search Trees via CSL

Mohammad Khan
Cornell University
msk295@cornell.edu

Zubin Bhaumik
Cornell University
zb57@cornell.edu

1 Introduction

The main strategy to solve chess with a computer engine involves the exploration of search trees representing all possible game states from the results of various moves and then choosing the move that leads to the best "average" game state. Since there is an exponentially large amount of possible game states as we simulate more moves, and deeper searches often lead to better results, it is imperative to use an algorithm to efficiently explore the search tree. One popular algorithm is Alpha-Beta pruning: a minimax-based search that effectively searches the whole tree, but eliminates suboptimal branches to increase efficiency. Another popular algorithm is Monte Carlo Tree Search (MCTS) which utilizes stochastic simulations to prioritize promising paths, but does not effectively search the whole tree.

Even with these strategies, the exponential nature of the search space limits how many moves deep we can explore. One can naturally guess that the next step to improve the depth of our search is parallelism (through multiple threads and multiple CPUs). However, parallel performance is bottlenecked in these algorithms by the irregular communication patterns and communication overhead. We hope that implementing these search strategies using the Cerebras Software Language (CSL) will improve their parallel performance due to the reduced communication overhead of their Wafer-Scale Engine. Our objective in this project will be to compare the parallel efficiency, computational depth, and competitive game performance of these algorithm when executing on Cerebras and traditional hardware.

2 Hypothesis

As mentioned above the question we are seeking to answer is whether WSE-2 implementations of Alpha-Beta pruning and Monte Carlo Tree Search will improve over their standard implementations across the three dimensions of parallel efficiency, computational depth, and competitive game performance.

In terms of parallel efficiency, we anticipate that standard MCTS will naturally outperform standard Alpha-Beta due to its reliance on simulations/rollouts which are more independent (and embarrassingly parallel) in nature compared to Alpha-Beta pruning which requires strict synchronization of search bounds (α and β) to remain effective (limit unnecessary work). However, we hypothesize that both algorithms will achieve significantly higher parallel efficiency on the WSE-2. The on-chip interconnect system should allow for low latency which is important for the relatively small message sizes (mostly just the search bounds) in Alpha-Beta and MCTS. We doubt that the improved throughput of the chip will be relevant, but the low latency should allow the search to scale to dozens/hundreds of processing elements (PE) with minimal communication stall (leading to less unnecessary searching)

Regarding computational depth and game performance, we don't expect to achieve better raw performance on WSE-2 (in fact we expect the opposite: magnitudes worse performance) because we are utilizing a software simulator rather than the physical WSE-2 hardware. However, if we implement some sort of normalization through clock cycles or FLOPs, we expect the WSE-2 implementations to reach greater depths and correspondingly demonstrate superior game performance per fixed amount of compute. This is because the WSE-2 can execute hundreds of localized search operations in the extra time that multiple CPUs/threads take to synchronize a broadcast operation for example.

In terms of the algorithm dimension, we expect that Alpha-Beta pruning will do much better in game performance (consistent with most chess engines like Stockfish using Alpha-Beta pruning over MCTS). This is because Alpha-Beta pruning effectively searches the whole tree, while MCTS and its stochastic nature means that certain vital moves could be missed. MCTS is usually used in games like Go with higher branching factor per move than Chess because searching whole layers of the search tree becomes unfeasible, so the stochastic penalty is not as strong. However, the gap between the two algorithms might be a bit closer than normal if the parallel efficiency of MCTS becomes much larger than Alpha-Beta pruning on the WSE-2 (more branches could be explored due to the higher efficiency limiting stochastic variance problems).

3 Context

We learned about Alpha-Beta pruning and MCTS in previous classes like CS 3/4700: Foundations of Artificial Intelligence which gave us the idea of implementing a parallel version for chess. However, this has already been done before, so we decided to shift our project towards replicating previous Alpha-Beta pruning and MCTS parallel results, and then comparing them to new results obtained from the WSE-2 system. Despite its focus not being on search related algorithm its low latency should provide a decent boost to our parallel implementations of both algorithms.

This project will solely be done by the two of us, and will not include collaboration with people outside of the course, but of course we will be using previous papers/implementations of Alpha-Beta and MCTS as reference. This work in not a subproject within a larger research effort, and it is also not be used in other classes. For identification purposes, we are Project Group 36 on Canvas.

4 Key Prior Work

Since the focus of our project is not creating a new parallel implementation of Alpha-Beta pruning or MCTS and instead comparing parallel implementations on standard hardware and WSE-2 hardware, we will be using previously done pseudocode and implementations to guide the development of our code. Some good options that stand out to us now are "Comparative Study of Performance of Parallel Alpha Beta Pruning for Different Architectures" by Singhal et al. [3] and "Parallelization of Monte Carlo Tree Search in

Continuous Domains" by Kurzer et al. [2] Note that while the Alpha-Beta pruning paper tests directly on chess within the paper, the MCTS paper is not directly applied to the chess environment. We think that despite this the paper is still a good resource because it talks about 3 different parallelization strategies and because MCTS isn't usually applied to chess anyway due to chess's relatively low branching factor so its difficult to find a direct resource.

Beyond just the insights into parallelized Alpha-Beta pruning and MCTS, there are still a significant amount of optimizations that can be done to speed up our implementation like bitboards. While we don't necessarily need these strategies for parallel efficiency comparisons, if we want to get performance close to SOTA so we can perform a competitive game performance test they are necessary. To help guide our implementation of these optimizations we will be using the Stockfish documentation [4]. Stockfish is considered to be the strongest chess engine in the world and is open source so it should be a great resource and later a benchmark (hopefully, we will at least be able to beat earlier versions of Stockfish). Finally, for all our CSL implementations, we will be heavily relying on the CSL documentation website, specifically the collective communication built in library [1].

5 Empirical Methodology

To investigate our hypothesis, we will follow the below steps. The first phase involves implementing Alpha-Beta pruning and MCTS on traditional CPU architectures (OpenMP and MPI), using various high-performance techniques such as bitboard board representations but also with relatively simple evaluation functions (like material advantage) to make sure they are more easily transferrable to CSL. Next, we will port these algorithms to CSL, with primary development priority given to Alpha-Beta due to its dominance in discrete chess search. Ideally, these algorithms should follow simple collective communication patterns like broadcast and reduce, but we might need more complicated distributed logic if memory requirements gets too large for a single PE (like splitting a hash map across groups of PEs).

To establish a rigorous comparison, the third phase focuses on scaling analysis. We will measure strong scaling by fixing the search depth of various chess positions and increasing processor counts, and weak scaling by increasing available processing elements as we increase the target search depth. For weak scaling we will likely have to calculate the total amount of nodes as we increase depth to figure out how many more processors we should include at each step. For the fourth phase, to normalize results, we will need to implement counters for clock cycles and FLOPs (we will use the one that normalizes results better or easier to implements). We can use cycle-limited rather than time-limited frameworks for our computational depth and competitive game performance comparisons. This approach allows us to determine if the WSE-2's low-latency interconnect leads to more efficient search per unit of compute than traditional MPI and OpenMP

Our computational requirements are highly scalable based on the desired search depth. However we hope to test our standard CPU implementations up to 2 Perlmutter nodes or 256 CPUs (so that both MPI and OpenMP) can be tested. We estimate that development should take less than 10 CPU hours (especially since a lot of development can be done locally), and that full testing will take somewhere between 10-20 CPU hours. As for our CSL code it will probably be run on login nodes (because of issues mentioned on EdStem from Homework 4), however we will make sure to limit our computational depth to not harm other user's experience. We can also try to test the CSL through local Podman containers if the login nodes are too limited.

6 Challenges and Obstacles

There are two main challenges that we can think of that might prevent us from investigating as far as we hope to. The first challenge being the difficulty of learning/using the CSL language. It was very difficult for us to implement synchronization for matrix multiplication during assignment four, so we are worried that we might run into similar issues during our implementations of Alpha-Beta and MCTS on WSE-2/CSL. We are a bit more hopeful because of two reasons: the synchronization requirements are a lot looser in these algorithms, if values aren't received in time the algorithms can still continue it would just do some unnecessary work. Secondly all the communication we need to do should just be collective communication operations like broadcast which have prebuilt implementations in CSL. If even with these advantages we find implementation on CSL difficult, we can then pivot our project to only implementing the simpler algorithm on CSL (probably Alpha-Beta pruning) and just go more in depth with our testing/analysis of this WSE-2 algorithm compared to the two standard algorithms.

The second challenge is more intimately related to competitive game performance goal mentioned in the methodology section. Specifically, we want to test our various parallel implementations versus a real chess bot like Stockfish (given a cycle or FLOP normalized environment). The problem with this comparison is that Stockfish not only has an efficient parallel implementation of Alpha-Beta pruning it also has a good heuristic/board rating function through an efficiently updatable neural network (NNUE). The problem is that this NNUE logic is too large (6 MB small to 130 MB large) to implement within a single PE. To fix this problem for our competitive game performance we have to use simpler heuristic functions to get accurate comparisons (the simplest being the material advantage function). This basically means using earlier versions of Stockfish (before 2021) which is a little disappointing but might be necessary. Hopefully like the above two challenges, any other challenges we run into can be solved through slight modifications that may lead away from SOTA but is worth it if we can get more valid comparisons.

References

[1] Cerebras Systems. 2026. *Cerebras Software Language (CSL) Documentation.* https://sdk.cerebras.net/ Accessed: 2026-03-27.

[2] Karl Kurzer, Christoph Hörtnagl, and J. Marius Zöllner. 2020. Parallelization of Monte Carlo Tree Search in Continuous Domains. arXiv:2003.13741 [cs.LG] https://arxiv.org/abs/2003.13741

[3] Shubhendra Pal Singhal and M. Sridevi. 2019. Comparative study of performance of parallel alpha Beta Pruning for different architectures. In *2019 IEEE 9th International Conference on Advanced Computing (IACC).* IEEE, 115–119. doi:10.1109/iacc48062.2019.8971591

[4] Stockfish Developers. 2024. Stockfish: A strong open-source chess engine. https://github.com/official-stockfish/Stockfish. Accessed: 2026-03-27.